Infrastructure as Code (IaC)

CSYE 6225 - Boston

Spring 2026

Tejas Parikh (t.parikh@northeastern.edu)

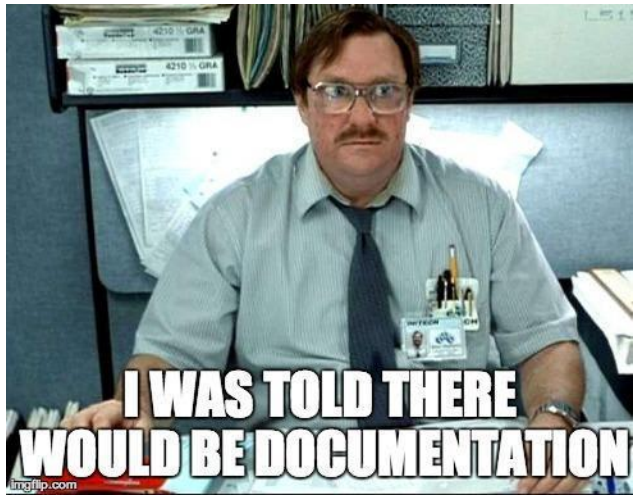
NORTHEASTERN.

Learning Objectives

- Explain what Infrastructure as Code is and why it exists.
- Identify the problems that arise when infrastructure is managed manually in dynamic cloud environments.
- Describe the core principles of IaC: reproducibility, disposability, consistency, repeatability, and continuous change.
- Differentiate between the "pets vs. cattle" mindset for managing servers.
- Define key IaC concepts such as stacks and environments.
- Articulate how version control systems support infrastructure management.

Why Infrastructure as Code?

- The promise of cloud computing and automation
- The reality: why cloud and automation often make things worse
- Server sprawl and the operational burden
- The gap between adopting tools and adopting effective practices
- Change management challenges in modern organizations



What Is Infrastructure as Code?

- Definition: infrastructure automation based on software development practices
- Emphasis on consistent, repeatable routines for provisioning and changing systems
- Treating infrastructure as software and data
- Connecting IaC to software development tools and practices: VCS, automated testing, TDD, CI/CD

Goals of Infrastructure as Code

- Infrastructure supports and enables change
- Routine changes without drama or stress
- Self-service resource provisioning
- Easy recovery from failures
- Continuous improvement over big-bang projects
- Proving solutions through implementation and measurement

Challenges with Dynamic Infrastructure

- Server sprawl
  - Rapid provisioning leads to unmanageable growth
  - Patching, updates, and vulnerability management at scale
- Configuration drift
  - How even consistently created servers diverge over time
  - Real-world scenarios: hotfixes, version mismatches, individual configurations
  - Managed vs. unmanaged variation
- Snowflake servers
  - Definition and why they are dangerous
  - The reproducibility test: can you rebuild this server right now?
- Fragile infrastructure
  - Snowflake problem at portfolio scale
  - The migration path to reproducible infrastructure

Principles of Infrastructure as Code

- Systems can be easily reproduced
  - Effortless and reliable rebuilding of any infrastructure element
  - Capturing all decisions in scripts and tooling
- Systems are disposable
  - Designing for constant change
  - Software must tolerate infrastructure appearing and disappearing
- Cattle, not pets
  - The pets vs. cattle analogy
  - Iron age (reliable hardware, unreliable software) vs. cloud age (reliable software, unreliable hardware)
- Systems are consistent
  - Near-identical infrastructure elements within a cluster
  - Consistency enables trust in automation
- Processes are repeatable
  - Scripting culture: if it can be scripted, script it
  - Scripts and configuration management over manual changes
- Design is always changing
  - Iron-age approach: comprehensive upfront design, expensive changes
  - Cloud-age approach: simple designs, frequent changes
  - Frequent changes force good habits, efficient processes, and supportive tooling

LVX
VERITAS
VIRTVS

Practicing Infrastructure as Code

- Stacks
  - Definition: a collection of infrastructure elements defined and changed as a unit
  - Stack sizing: from a single server to an entire data center
  - Organizing stacks beyond network boundaries
- Environments
  - Development, test, preproduction, production
  - Keeping environments consistent with IaC
- Version control for infrastructure management
  - VCS as the core of an IaC regime
  - Traceability, rollback, correlation, visibility, and automated triggers
- What to manage in VCS
  - Configuration definitions (cookbooks, manifests, playbooks)
  - Configuration files, templates, test code, CI/CD job definitions, utility scripts, documentation
- What NOT to manage in VCS
  - Software artifacts (use artifact repositories)
  - Built artifacts that can be rebuilt from source
  - Application data, logs, and backups
  - Passwords and security secrets (use secrets management tools)

Key Terms

Term	Definition
Infrastructure as Code (IaC)	Managing and provisioning infrastructure through machine-readable definition files rather than manual processes
Configuration Drift	Gradual divergence of infrastructure configurations from their intended or original state
Snowflake Server	A server that is unique and cannot be reliably reproduced
Fragile Infrastructure	An infrastructure portfolio that is easily disrupted and difficult to repair
Stack	A collection of infrastructure elements defined and managed as a single unit
Environment	An instance of a stack, such as development, staging, or production
Idempotency	The property that applying the same operation multiple times produces the same result

Additional Resources

See course lecture page on Canvas for supplementary materials and documentation links.

LVX
VERITAS
VIRTVS